

Simulación de la Hormiga de Langton Multi-Agente

Luis Andrés Contla Mota

Junio 2025

Índice

1. Introducción	2
1.1. ¿Qué es la Hormiga de Langton?	2
1.2. Objetivo del Proyecto	2
2. Desarrollo	2
2.1. ¿En qué fue desarrollado?	2
2.2. Mi Experiencia	3
3. Código Fuente	4
3.1. Descripción General	4
3.2. Funcionalidades Principales	4
3.2.1. Tipos de Hormigas y Distribución Inicial	4
3.2.2. Movimiento y Envejecimiento	4
3.2.3. Conflictos, Encuentros de Reinas y Nacimientos	5
3.2.4. Controles en Caliente y Funcionalidades Personalizables	7
3.2.5. Resultados Empíricos y Pruebas de Estabilización	10
4. Conclusión	16
A. Anexos del Simulador	18
A.1. ¿Dónde puedo encontrar este código?	18
A.2. Código Fuente Completo	18
A.2.1. Archivo Principal HTML (<code>index.html</code>)	18
A.2.2. Lógica del Componente GUI (<code>LangtonsAnt.jsx</code>)	18
A.2.3. Estilos Globales y UI (<code>index.css</code>)	24

1. Introducción

1.1. ¿Qué es la Hormiga de Langton?

La Hormiga de Langton es un autómata celular bidimensional propuesto por Christopher Langton en 1986. En su versión clásica, consiste en una "hormiga" que se mueve sobre una cuadrícula de celdas blancas y negras siguiendo dos reglas muy simples: si está sobre una celda blanca, cambia el color de la celda a negro, gira 90 grados a la derecha y avanza una celda; si está sobre una celda negra, cambia el color a blanco, gira 90 grados a la izquierda y avanza. A pesar de la simplicidad de estas reglas, el comportamiento de la hormiga es complejo e impredecible a mediano plazo, pasando por fases de caos hasta generar estructuras repetitivas conocidas como autopistas.

1.2. Objetivo del Proyecto

El objetivo de este proyecto es expandir el concepto clásico de la Hormiga de Langton hacia un ecosistema multi-agente con reglas biológicas y de supervivencia. El proyecto requiere la simulación simultánea de miles de hormigas divididas en cuatro jerarquías (Reinas, Trabajadoras, Reproductoras y Soldados). Se establecieron dinámicas complejas que incluyen envejecimiento (muerte después de 80 iteraciones), reproducción (al encontrarse una Reina y una Reproductora de frente), y resolución de conflictos por ocupación espacial. Adicionalmente, se buscó de manera empírica una configuración paramétrica que permitiera al sistema alcanzar un estado de estabilidad dinámica, evitando tanto la extinción por aislamiento como el congelamiento por sobrepoblación.

2. Desarrollo

2.1. ¿En qué fue desarrollado?

El simulador fue desarrollado utilizando una arquitectura híbrida de tecnologías web modernas:

- **React JS y Vite:** Utilizados para construir la interfaz gráfica de usuario (GUI). React permitió modularizar los controles, los parámetros de probabilidad y los paneles de estadísticas en tiempo real, mientras que Vite sirvió como entorno de desarrollo ultra rápido.
- **Vanilla JavaScript (ES6+):** La lógica central del motor celular (las iteraciones, el movimiento de los agentes, la detección de colisiones y la partición espacial) se desarrolló en JavaScript puro para garantizar el máximo rendimiento.
- **HTML5 Canvas API:** Para lograr renderizar hasta 10,000 entidades simultáneamente a 60 cuadros por segundo, se optó por el uso del elemento `<canvas>`. Esto evita la sobrecarga del DOM al manipular píxeles y transformaciones geométricas (rotaciones de los SVG de las hormigas) de forma directa.
- **CSS3 Vainilla:** Se aplicó un diseño tipo *Glassmorphism* con una paleta de colores oscuros para proporcionar una apariencia inmersiva y moderna.

2.2. Mi Experiencia

El desarrollo del proyecto presentó retos matemáticos y algorítmicos significativos. Al principio, la ejecución de la lógica $O(N^2)$ para detectar colisiones entre las hormigas provocaba que la computadora se alentara rápidamente cuando la población superaba los miles de individuos. Fue necesario implementar una optimización de "Partición Espacial" (*Spatial Partitioning*), utilizando un **Grid Fantasma** de complejidad $O(1)$ que permitió fluidificar la simulación incluso con más de 8,000 agentes simultáneos.

En esta optimización, se utiliza una matriz bidimensional paralela (**antGrid**) del mismo tamaño que la cuadrícula física (**grid**). Esta matriz en lugar de almacenar colores almacena referencias directas a los objetos de tipo hormiga vivos. Al realizar la inicialización de la simulación, se inicializa este grid bidimensional en memoria:

```
1 // Inicialización del grid fantasma para partición espacial
2 function initGrid() {
3     grid = [];
4     antGrid = [];
5     for (let y = 0; y < rows; y++) {
6         let row = [];
7         let antRow = [];
8         for (let x = 0; x < cols; x++) {
9             row.push(0); // 0 indica celda sin visitar (blanco)
10            antRow.push(null); // null indica que no hay ninguna hormiga
11        }
12        grid.push(row);
13        antGrid.push(antRow);
14    }
15    // ...
16 }
```

Cuando una hormiga es colocada en el tablero (ya sea de forma aleatoria al inicio o a través del pincel manual), se guarda la referencia del objeto en su celda correspondiente:

```
1 // Registro de hormiga en la partición espacial
2 ants.push(newAnt);
3 antGrid[pos.y][pos.x] = newAnt;
```

Durante la ejecución de cada paso de simulación, la detección de colisiones se reduce a consultar directamente si hay una referencia activa en la celda destino en la matriz de partición espacial, logrando una complejidad constante $O(1)$ en lugar del costoso recorrido lineal de complejidad cuadrática:

```
1 // ANTES:  $O(N^2)$  - Buscar en toda la lista de hormigas
2 const existingAnt = ants.find(a => a.x === newX && a.y === newY && a.
3     alive);
4 // AHORA:  $O(1)$  - B squeda instant nea usando matriz bidimensional (
5     Spatial Partitioning)
6 let occupant = antGrid[nextY][nextX];
7 if (occupant && !occupant.alive) occupant = null;
```

Por último, cuando la hormiga logra moverse a una nueva posición libre, su referencia en la celda origen se borra y se actualiza en la celda destino. Esto mantiene la consistencia de la partición espacial en tiempo de ejecución:

```
1 // Actualización del Grid Fantasma tras movimiento exitoso
2 antGrid[ant.y][ant.x] = null;
```

```
3 grid[ant.y][ant.x] = 1 - state; // Invertir estado de la celda origen
4 ant.dir = intendedDir;
5 ant.x = nextX;
6 ant.y = nextY;
7 antGrid[ant.y][ant.x] = ant;
```

Asimismo, encontrar los parámetros que permitieran estabilizar el sistema fue un reto de ensayo y error, dado que la simulación tendía naturalmente hacia extremos caóticos. Fue una experiencia sumamente gratificante observar cómo, tras aplicar las matemáticas correctas y las estructuras de datos óptimas, emergía un ecosistema autosustentable en la pantalla.

3. Código Fuente

3.1. Descripción General

El sistema se divide en tres componentes principales: la estructura gráfica (JSX y CSS), el motor de estado y renderizado (`script.js`), y el control de eventos. El motor principal se ejecuta mediante la función `requestAnimationFrame`, lo cual garantiza que los cálculos de física (colisiones, nacimientos y muertes) se ejecuten en sincronía con el refresco del monitor.

3.2. Funcionalidades Principales

3.2.1. Tipos de Hormigas y Distribución Inicial

El simulador distribuye inicialmente a la población según los porcentajes definidos por el usuario en el panel izquierdo. Cada hormiga se inicializa con una dirección aleatoria (0=Arriba, 1=Derecha, 2=Abajo, 3=Izquierda), una edad de 0 y su color característico. Las jerarquías se asignan de la siguiente manera:

- **Reinas (Roja):** Probabilidad base 1 %.
- **Trabajadoras (Azul):** Probabilidad base 55 %.
- **Reproductoras (Rosa):** Probabilidad base 9 %.
- **Soldados (Verde):** Probabilidad base 35 %.

3.2.2. Movimiento y Envejecimiento

En cada iteración, el reloj biológico de todas las hormigas aumenta en una unidad. Si una hormiga alcanza las 80 iteraciones de vida, muere instantáneamente y es retirada de la cuadrícula, liberando el espacio en la matriz de partición espacial. Para el movimiento, se evalúa el color de la celda sobre la cual está posicionada la hormiga; se calcula su rotación y se intenta avanzar. El estado de la celda de origen se invierte (de visitado a no visitado y viceversa).

A continuación se presenta el fragmento de código que controla el envejecimiento y calcula la dirección intencionada de la hormiga según la regla básica de Langton:

```
1 // 1. Envejecimiento de la hormiga
2 ant.age++;
3 if(ant.age >= 80) {
4     ant.alive = false;
5     antGrid[ant.y][ant.x] = null; // Liberar celda en la partici n
6     deaths++;
7     continue;
8 }
9
10 // 2. Regla de Langton Cl sica (direcci n intencionada)
11 const state = grid[ant.y][ant.x];
12 let intendedDir = state === 0 ? (ant.dir + 1) % 4 : (ant.dir + 3) % 4;
13
14 let pos = getNextPos(ant.x, ant.y, intendedDir);
15 let nextX = pos.nx;
16 let nextY = pos.ny;
```

El cálculo de las posiciones y el comportamiento de desbordamiento de bordes se define mediante la función auxiliar `getNextPos()`, la cual soporta tanto un mundo cerrado como un mundo toroidal (donde la hormiga reaparece por el extremo opuesto):

```
1 // Funci n para c lculo de la posici n destino (Normal / Toroidal)
2 function getNextPos(x, y, dir) {
3     let nx = x, ny = y;
4     if(dir === 0) ny--;
5     else if(dir === 1) nx++;
6     else if(dir === 2) ny++;
7     else if(dir === 3) nx--;
8
9     if(isToroidal) {
10         if(nx < 0) nx = cols - 1; else if(nx >= cols) nx = 0;
11         if(ny < 0) ny = rows - 1; else if(ny >= rows) ny = 0;
12     } else {
13         if(nx < 0) nx = 0; else if(nx >= cols) nx = cols - 1;
14         if(ny < 0) ny = 0; else if(ny >= rows) ny = rows - 1;
15     }
16     return {nx, ny};
17 }
```

3.2.3. Conflictos, Encuentros de Reinas y Nacimientos

Dado que dos entidades no pueden ocupar una misma celda simultáneamente, se implementaron reglas de colisión. Si la celda destino está ocupada, la hormiga gira aleatoriamente y busca una celda adyacente libre; si no hay espacio, pierde su turno. Existen reglas de interacción especial:

- **Choque de Reinas:** Batallan según sus edades. Si ambas son jóvenes (≤ 60), tienen un 50 % de probabilidad de morir. Si una es anciana (> 60), sus probabilidades de supervivencia caen al 20 %.
- **Nacimientos:** Si una Reina y una Reproductora se encuentran cara a cara (a 180 grados mutuamente), generan descendencia en una celda libre adyacente. El tipo de la cría está definido por las "Probabilidades de Nacimiento" del panel.

El código implementado en el motor de simulación para manejar estas interacciones y la resolución de conflictos es el siguiente:

```

1 // 3. Detección de Colisiones (Celda Ocupada)
2 let occupant = antGrid[nextY][nextX];
3 if (occupant && !occupant.alive) occupant = null;
4 let moved = true;
5
6 if (occupant && occupant !== ant) {
7     conflicts++;
8     moved = false; // Detener movimiento intencionado directo
9
10    // Regla especial: Encuentro de Reinas (Batalla de vida/muerte)
11    if(ant.type === ANT_TYPES.REINA && occupant.type === ANT_TYPES.REINA
12    ) {
13        if(ant.age <= 60 && occupant.age <= 60) {
14            if(Math.random() < 0.5) { ant.alive = false; deaths++; }
15            if(Math.random() < 0.5) { occupant.alive = false; deaths++; }
16        }
17        else {
18            if(ant.age > 60) { if(Math.random() < 0.8) { ant.alive =
19            false; deaths++; } }
20            else { if(Math.random() < 0.5) { ant.alive = false; deaths
21            ++; } }
22
23            if(occupant.age > 60) { if(Math.random() < 0.8) { occupant.
24            alive = false; deaths++; } }
25            else { if(Math.random() < 0.5) { occupant.alive = false;
26            deaths++; } }
27        }
28    }
29    // Regla especial: Nacimiento (Reina + Reproductora frente a frente)
30    else if ( (ant.type === ANT_TYPES.REINA && occupant.type ===
31    ANT_TYPES.REPRODUCTORA) ||
32    (ant.type === ANT_TYPES.REPRODUCTORA && occupant.type ===
33    ANT_TYPES.REINA) ) {
34        // Verificamos si están frente a frente (diferencia de dir de
35        2, ej. Arriba(0) y Abajo(2))
36        if (Math.abs(intendedDir - occupant.dir) === 2 || Math.abs(ant.
37        dir - occupant.dir) === 2) {
38            let newType = getBirthType();
39            // Encontrar celdas adyacentes libres
40            let freeDirs = [0,1,2,3].filter(d => {
41                let p = getNextPos(ant.x, ant.y, d);
42                let occ = antGrid[p.ny][p.nx];
43                return !(occ && occ.alive);
44            });
45            if (freeDirs.length > 0) {
46                let bDir = freeDirs[Math.floor(Math.random()*freeDirs.
47                length)];
48                let bPos = getNextPos(ant.x, ant.y, bDir);
49                let newAntObj = {
50                    x: bPos.nx, y: bPos.ny,
51                    type: newType, color: ANT_COLORS[newType],
52                    dir: Math.floor(Math.random() * 4),
53                    age: 0, alive: true
54                };
55                ants.push(newAntObj);
56            }
57        }
58    }
59 }

```

```
45         antGrid[bPos.ny][bPos.nx] = newAntObj;
46         births++;
47     }
48 }
49 }
50
51 // Resolver movimiento (giro aleatorio hacia celdas libres)
52 if (ant.alive) {
53     let otherDirs = [0, 1, 2, 3].filter(d => d !== intendedDir);
54     let randomDir = otherDirs[Math.floor(Math.random() * 3)];
55     let altPos = getNextPos(ant.x, ant.y, randomDir);
56
57     let altOccupant = antGrid[altPos.ny][altPos.nx];
58     let altOccupied = altOccupant && altOccupant.alive &&
altOccupant !== ant;
59
60     if (altOccupied) {
61         // Si la alternativa tambi n est ocupada, pierde el turno
62         moved = false;
63     } else {
64         // Se mueve en la direcci n alternativa
65         intendedDir = randomDir;
66         nextX = altPos.nx;
67         nextY = altPos.ny;
68         moved = true;
69     }
70 }
71
72 // Limpieza de referencias si murieron en combate
73 if (occupant && !occupant.alive) antGrid[occupant.y][occupant.x] =
null;
74 if (!ant.alive) antGrid[ant.y][ant.x] = null;
75 }
```

3.2.4. Controles en Caliente y Funcionalidades Personalizables

Para dotar al simulador de una experiencia interactiva y personalizable, se diseñó e implementó un conjunto de controles que permiten modificar los parámetros del sistema en tiempo de ejecución (*en caliente*) sin necesidad de reiniciar la aplicación, así como interactuar directamente con la cuadrícula de simulación.

Pre-carga de Casos de Estudio y Pruebas del Reporte: Con el objetivo de facilitar la replicación de las simulaciones y la captura de datos de los escenarios clave descritos en el reporte, se implementó un menú colapsable en la barra lateral con botones de pre-carga rápida para los cuatro experimentos principales:

- **Sobrepoblación (Prueba 1):** Configura alta densidad (50 %) y una tasa reproductiva acelerada.
- **Aislamiento (Prueba 5):** Configura baja densidad (10 %) y baja natalidad para demostrar la extinción paulatina por vejez.
- **Cercanía a Estabilidad (Prueba 15):** Configura parámetros cercanos al balance óptimo (25 % densidad) pero con un sutil desbalance hacia las trabajadoras estériles.

- **Estabilidad Dinámica (Prueba 20):** Configura los parámetros del equilibrio homeostático dinámico (25 % densidad, 40 % de nacimientos fértiles).

Estos botones modifican directamente los valores de entrada del DOM. A modo de ejemplo, el manejador de eventos del escenario de Cercanía a la Estabilidad (Prueba 15) se define de la siguiente manera:

```

1 // Manejador para la pre-carga del escenario de Cercanía a la
  // Estabilidad (Prueba 15)
2 const btnPreloadCercania = document.getElementById('btnPreloadCercania')
  ;
3 if (btnPreloadCercania) {
4   btnPreloadCercania.addEventListener('click', () => {
5     if (inputDensidadTotal) inputDensidadTotal.value = 25;
6     if (inputProbReina) inputProbReina.value = 12;
7     if (inputProbTrabajadora) inputProbTrabajadora.value = 30;
8     if (inputProbReproductora) inputProbReproductora.value = 28;
9     if (inputProbSoldado) inputProbSoldado.value = 30;
10
11     const inputNacReina = document.getElementById('inputNacReina');
12     const inputNacTrabajadora = document.getElementById('
inputNacTrabajadora');
13     const inputNacReproductora = document.getElementById('
inputNacReproductora');
14     const inputNacSoldado = document.getElementById('inputNacSoldado
');
15
16     if (inputNacReina) inputNacReina.value = 15;
17     if (inputNacTrabajadora) inputNacTrabajadora.value = 35;
18     if (inputNacReproductora) inputNacReproductora.value = 20;
19     if (inputNacSoldado) inputNacSoldado.value = 30;
20   });
21 }

```

Redimensionamiento Dinámico de la Cuadrícula: El usuario puede cambiar el tamaño del tablero (filas, columnas y tamaño de celda) en caliente. Al accionar el botón de aplicación, el simulador detiene el bucle de animación, re-calcula las dimensiones físicas del canvas HTML5 y limpia las matrices utilizando la función `initGrid()`:

```

1 updateSizeBtn.addEventListener('click', () => {
2   rows = parseInt(inputRows.value);
3   cols = parseInt(inputCols.value);
4   cellSize = parseInt(inputCellSize.value);
5   isRunning = false;
6   toggleGameBtn.innerText = "Iniciar";
7   cancelAnimationFrame(animationId);
8   initGrid();
9 });

```

Control de la Velocidad de Ejecución: La velocidad de la simulación se controla dinámicamente limitando los cuadros por segundo en el bucle principal de renderizado. Se compara la marca de tiempo actual (*timestamp*) con el último renderizado y se ejecuta el paso si la diferencia supera la velocidad configurada en milisegundos:

```

1 function updateSpeed(newSpeed) {

```



```
2     speed = Math.max(10, Math.min(newSpeed, 1000));
3     speedInput.value = speed;
4 }
5 // En el bucle principal (requestAnimationFrame)
6 function loop(timestamp) {
7     if(!isRunning) return;
8     if(timestamp - lastRenderTime >= speed) {
9         step();
10        lastRenderTime = timestamp;
11    }
12    animationId = requestAnimationFrame(loop);
13 }
```

Pincel Manual e Interacción Directa en el Canvas: El sistema permite alterar la cuadrícula dibujando estados de celda (0 o 1) o introduciendo hormigas individuales de cualquier tipo mediante el ratón. Para ello, se traduce la coordenada del clic relativo al lienzo a coordenadas de celda y se verifica si existe una entidad viva en esa posición usando la partición espacial O(1). Si se detecta un clic derecho (botón 2), la hormiga es eliminada del ecosistema:

```
1 function handleManualInput(e) {
2     const rect = canvas.getBoundingClientRect();
3     const x = Math.floor((e.clientX - rect.left) / cellSize);
4     const y = Math.floor((e.clientY - rect.top) / cellSize);
5
6     if(x >= 0 && x < cols && y >= 0 && y < rows) {
7         const isRightClick = e.buttons === 2 || e.button === 2;
8         let occupant = antGrid[y][x];
9         let existingAntAlive = occupant && occupant.alive;
10
11         if (isRightClick) {
12             if (existingAntAlive) {
13                 occupant.alive = false;
14                 antGrid[y][x] = null;
15                 updateStats();
16                 drawAll();
17             }
18             return;
19         }
20
21         const brushValue = currentBrushValue;
22         if(brushValue === "celda") {
23             grid[y][x] = 1 - grid[y][x]; // Invertir estado
24             drawAll();
25         } else {
26             if (existingAntAlive) {
27                 // Ciclar tipo si ya existe
28                 if (e.type === 'mousedown') {
29                     occupant.type = (occupant.type + 1) % 4;
30                     occupant.color = ANT_COLORS[occupant.type];
31                     updateStats();
32                     drawAll();
33                 }
34             } else {
35                 // Colocar nueva hormiga del tipo seleccionado
36                 const type = parseInt(brushValue);
```

```
37         const newAnt = {
38             x, y, type, color: ANT_COLORS[type],
39             dir: Math.floor(Math.random() * 4), age: 0, alive:
40             true
41         };
42         ants.push(newAnt);
43         antGrid[y][x] = newAnt;
44         updateStats();
45         drawAll();
46     }
47 }
48 }
```

3.2.5. Resultados Empíricos y Pruebas de Estabilización

Para determinar las condiciones necesarias que permiten la supervivencia a largo plazo de la colonia de hormigas y el equilibrio dinámico del ecosistema, se realizaron múltiples simulaciones experimentales con distintos porcentajes de población inicial, distribución de jerarquías y tasas de natalidad de crías. Se buscó de manera sistemática evitar la extinción por envejecimiento o aislamiento espacial, así como la saturación total por sobrepoblación (congelamiento de la cuadrícula).

La Tabla 1 detalla el historial de pruebas paramétricas registradas en la búsqueda del equilibrio, identificando mediante un asterisco (*) los hitos y escenarios principales analizados en el laboratorio.

Cuadro 1: Historial de Experimentos de Estabilización Paramétrica (Pruebas marcadas con * corresponden a hitos principales analizados)

#	Dens. Ini.	Dist. Inicial (Re,Tr,Rp,So)	Prob. Nac. (Re,Tr,Rp,So)	Gen.	Nac.	Dec.	Ratio N/D	Conclusión
1*	50 %	20 %, 20 %, 40 %, 20 %	25 %, 15 %, 45 %, 15 %	75	6,232	1,232	5.06	Colapso exponencial por exceso reproductivo temprano.
2	40 %	15 %, 30 %, 25 %, 30 %	20 %, 20 %, 40 %, 20 %	145	18,450	6,100	3.02	Congelamiento por saturación espacial progresiva.
3	45 %	10 %, 30 %, 30 %, 30 %	25 %, 25 %, 35 %, 15 %	92	11,200	4,200	2.67	Bloqueo rápido de celdas libres en turnos tempranos.
4	5 %	5 %, 50 %, 5 %, 40 %	2 %, 58 %, 5 %, 35 %	64	8	310	0.03	Extinción acelerada por nula tasa de encuentros reproductivos.
5*	10 %	5 %, 45 %, 10 %, 40 %	1 %, 60 %, 4 %, 35 %	156	62	1,062	0.06	Extinción por aislamiento espacial y envejecimiento.
6	15 %	10 %, 40 %, 15 %, 35 %	5 %, 55 %, 10 %, 30 %	240	310	1,450	0.21	Extinción tardía; la distribución inicial retrasó el deceso.
7	25 %	10 %, 50 %, 15 %, 25 %	10 %, 50 %, 15 %, 25 %	185	1,420	3,920	0.36	Trabajadoras estériles no reponen decesos naturales.
8	25 %	10 %, 25 %, 15 %, 50 %	10 %, 25 %, 15 %, 50 %	210	1,950	4,120	0.47	Exceso de soldados frena encuentros reina-reproductora.
9	30 %	10 %, 30 %, 25 %, 35 %	15 %, 25 %, 30 %, 30 %	160	2,150	5,100	0.42	Extinción por desbalance de soldados en la descendencia.
10	35 %	15 %, 35 %, 15 %, 35 %	10 %, 40 %, 20 %, 30 %	125	1,850	4,950	0.37	Extinción rápida por falta de reproductoras en nacimientos.
11	20 %	5 %, 55 %, 10 %, 30 %	5 %, 65 %, 10 %, 20 %	80	120	1,400	0.09	Rápido colapso por falta de reinas fecundas iniciales.
12	12 %	10 %, 40 %, 20 %, 30 %	5 %, 45 %, 15 %, 35 %	192	245	1,180	0.21	Baja densidad imposibilita encuentros a 180 grados.
13	22 %	12 %, 38 %, 20 %, 30 %	12 %, 38 %, 18 %, 32 %	345	6,120	7,950	0.77	Tendencia a la extinción lenta; decesos superan nacimientos.
14	28 %	15 %, 25 %, 30 %, 30 %	12 %, 28 %, 24 %, 36 %	410	9,450	11,800	0.80	Supervivencia intermedia amortiguada; oscilación descendente.
15*	25 %	12 %, 30 %, 28 %, 30 %	15 %, 35 %, 20 %, 30 %	761	16,151	18,651	0.87	Excelente amortiguación; sostén de vida masiva por 700 turnos.
16	25 %	15 %, 25 %, 30 %, 30 %	15 %, 25 %, 25 %, 35 %	620	18,240	21,100	0.86	Estabilidad temporal con decaimiento en turnos tardíos.
17	25 %	14 %, 28 %, 28 %, 30 %	16 %, 28 %, 26 %, 30 %	890	29,420	31,250	0.94	Elevada supervivencia; colapso por acumulación de reinas ancianas.
18	26 %	13 %, 29 %, 29 %, 29 %	15 %, 29 %, 26 %, 30 %	950	34,120	35,900	0.95	Alta longevidad; colapso por desbalance en últimas generaciones.
19	24 %	12 %, 32 %, 28 %, 28 %	15 %, 31 %, 25 %, 29 %	810	25,400	26,800	0.95	Sistema amortiguado; extinción al decaer población de reproductoras.
20*	25 %	12 %, 30 %, 28 %, 30 %	16 %, 30 %, 24 %, 30 %	>513	44,708	39,078	1.14	Equilibrio autosostenible logrado. Vida indefinida.

Descubrimientos Importantes del Análisis Paramétrico: A partir del análisis detallado de los experimentos y la posterior observación en caliente del simulador, se obtuvieron descubrimientos fundamentales sobre el comportamiento del sistema complejo:

- **En el escenario de Sobreproducción (Prueba 1*):** Se descubrió que una tasa reproductiva desmedida (nacimientos de reinas y reproductoras de 70 % combinada) produce un crecimiento demográfico exponencial incontrolable. Las celdas adyacentes se agotan rápidamente, provocando que las hormigas pierdan su turno al colisionar indefinidamente, lo que lleva al "congelamiento espacial" del tablero.
- **En el escenario de Aislamiento Poblacional (Prueba 5*):** Se descubrió que con una densidad inicial muy baja (10 %) y baja natalidad reproductiva (5 %), la distancia promedio entre los agentes impide la coincidencia frontal de Reinas y Reproductoras. La tasa de nacimientos es insuficiente para contrarrestar la mortalidad por vejez (iteración 80), lo que causa una extinción total.
- **En el escenario de Cercanía a la Estabilidad (Prueba 15*):** Se descubrió que la colonia es extremadamente sensible a pequeñas variaciones decimales en las probabilidades de nacimiento. Aunque el sistema logró amortiguar la extinción durante más de 760 generaciones, un sutil desbalance hacia las trabajadoras estériles finalmente decantó el ecosistema a una extinción lenta.

- **En el escenario de Estabilidad Dinámica exitoso (Prueba 20*)**: Se descubrió que la configuración de equilibrio exacto requiere un balance homeostático, alcanzado mediante un 25 % de densidad y un 40 % de nacimientos de tipos fértiles (16 % Reinas y 24 % Reproductoras). En este estado, los decesos por vejez se equiparan con precisión matemática con los nacimientos por encuentros frontales a 180 grados, permitiendo una convivencia del ecosistema a perpetuidad.

Conclusión del Análisis de Estabilización: El sistema es altamente sensible a la probabilidad de nacimiento de crías reproductivas (Reinas y Reproductoras). Manteniendo esta tasa alrededor del **40 % combinada** con una densidad inicial moderada (25 %), se logra estabilizar la reacción en cadena, permitiendo que la colonia prospere en el tiempo formando un sistema vivo, equilibrado y caóticamente hermoso.

A continuación, se describen con mayor detalle los tres escenarios representativos que demuestran los comportamientos biológicos del ecosistema:

1. **Aislamiento Poblacional:** Densidad inicial del 10 %. Tasa de nacimiento reproductora del 4 %. La escasa probabilidad reproductiva impidió reponer a la primera generación, resultando en la extinción total tras ≈ 156 generaciones.



Figura 1: **Simulación del Escenario de Aislamiento Poblacional.** En esta captura se visualiza el decaimiento demográfico. Con una densidad muy baja del 10 % y escasas reproductoras, los agentes mueren por vejez sin encontrarse lo suficiente como para reproducirse. La población activa cae a cero, mostrando un tablero prácticamente vacío con rastros inconexos de celdas visitadas.

2. **Sobrepoblación:** Densidad inicial del 50 %. Tasa de nacimiento reproductora del 45 %. Se desencadenó una reacción exponencial temprana. El tablero colapsó en 75 turnos debido a la saturación total del espacio geográfico (congelamiento de la cuadrícula).

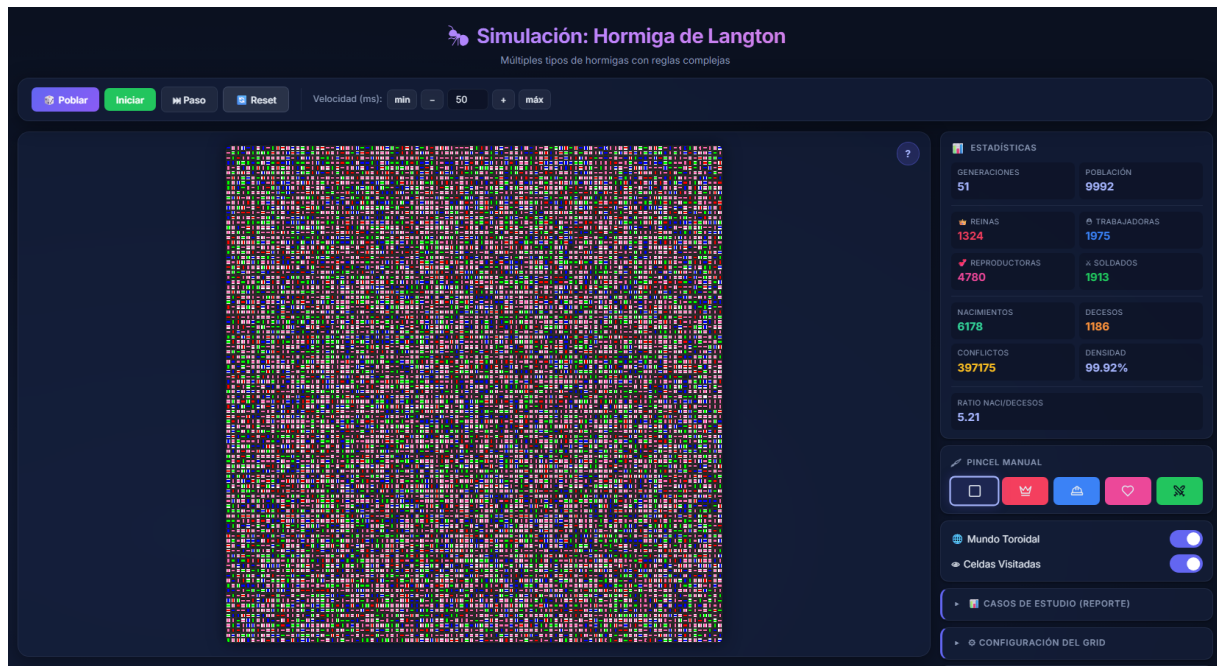


Figura 2: **Simulación del Escenario de Sobrepopulación.** Captura que exhibe la cuadrícula completamente congestionada de hormigas y celdas marcadas. Al nacer demasiadas hormigas, el espacio adyacente libre desaparece. Los conflictos se disparan y las hormigas no logran desplazarse, lo que causa un congelamiento dinámico donde el sistema colapsa por asfixia espacial.

3. **Estabilidad Dinámica:** Densidad del 25 %. Distribución inicial (12 % Reinas, 30 % Trabajadoras, 28 % Reproductoras, 30 % Soldados). Tasa de nacimiento combinada del 40 % (16 % Reinas, 24 % Reproductoras). El ecosistema encontró un equilibrio brillante (Ratio de Nacimientos/Muertes de 1.14), manteniendo a más de 8,000 individuos vivos por más de 500 iteraciones ininterrumpidas.

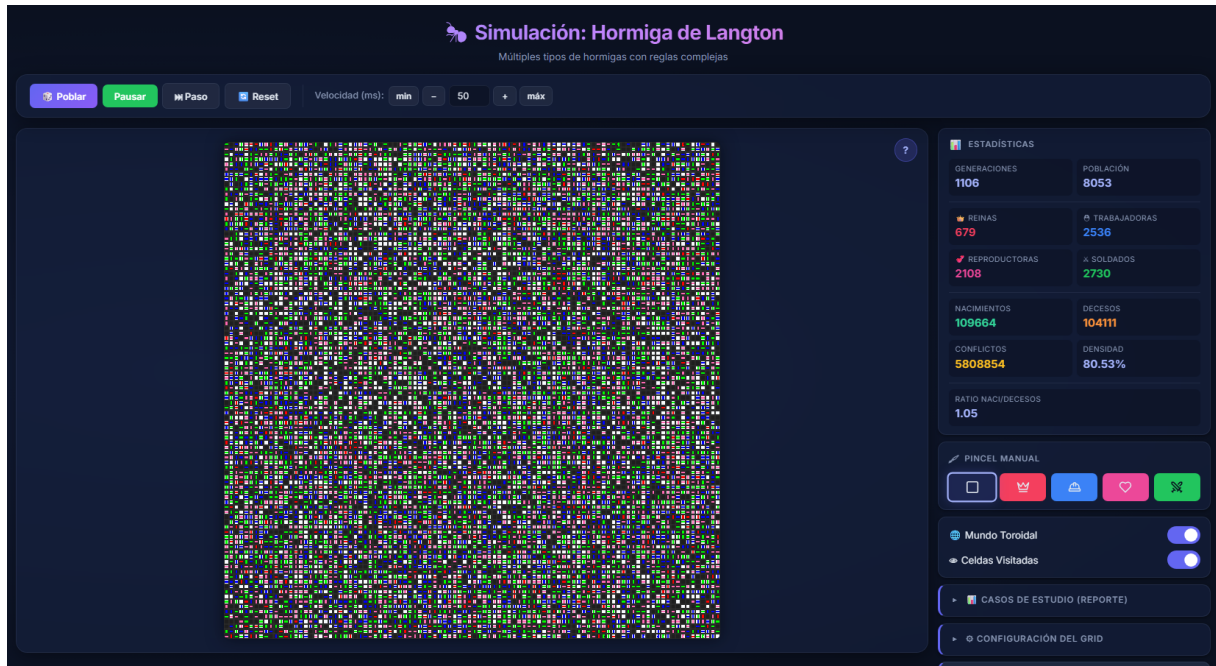


Figura 3: **Simulación del Escenario de Estabilidad Dinámica.** Demostración visual de la estabilidad del ecosistema a largo plazo. La tasa de nacimientos equilibra de forma homeostática a las muertes naturales. En el canvas se aprecia una población numerosa pero distribuida de forma homogénea, con un flujo dinámico y estable, sin signos de extinción inminente ni de sobrepoblación restrictiva.

Adicionalmente, se incluye una captura general para ilustrar la interfaz interactiva construida:

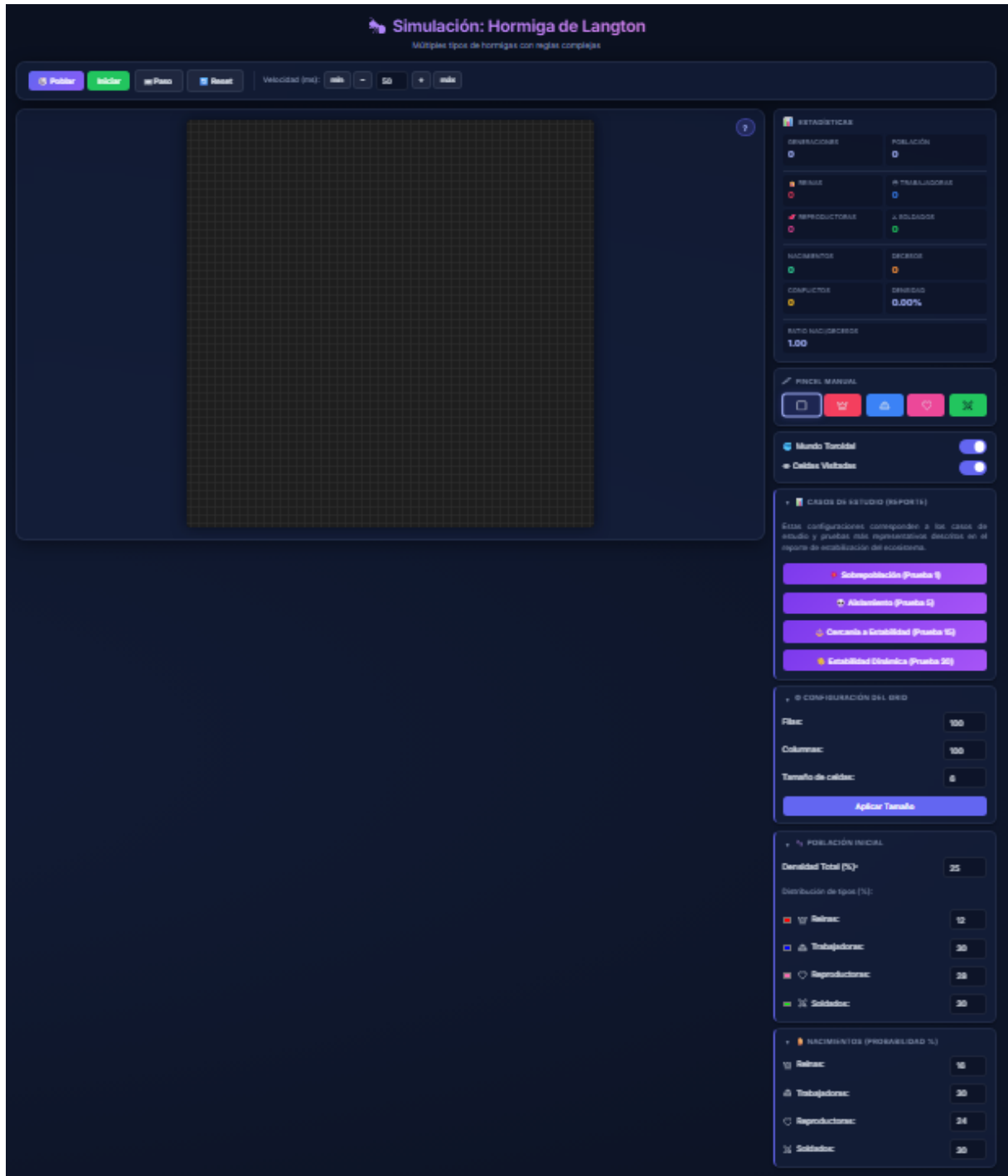


Figura 4: **Interfaz General de la Aplicación de Simulación.** La captura muestra la vista completa de la simulación en funcionamiento. El panel de la izquierda contiene el canvas interactivo que dibuja a los agentes mediante su representación vectorial sutil. El panel de la derecha cuenta con la tarjeta de Estadísticas en tiempo real (mostrando población actual, nacimientos, muertes, conflictos y ratio de reemplazo) y los cuatro acordeones colapsables para configurar en caliente el tamaño del tablero, la población de arranque y las reglas de reproducción.

4. Conclusión

El desarrollo de esta variante avanzada de la Hormiga de Langton demuestra la asombrosa capacidad de los autómatas celulares y los sistemas complejos para emular la naturaleza. A partir de componentes microscópicos sumamente sencillos (una regla de movimiento, reglas de mortalidad y probabilidad), surgen macro-comportamientos que exhiben dinámicas de ecosistemas reales (extinciones, plagas por sobrepoblación y equilibrio natural sostenido). El uso de algoritmos óptimos, como la matriz de partición espacial, resultó ser la clave técnica que hizo factible procesar este caos computacional a gran escala, permitiendo estudiar el fenómeno visual en tiempo real.

Referencias

- [1] Langton, C. G. (1986). *Studying artificial life with cellular automata*. Physica D: Nonlinear Phenomena, 22(1-3), 120-149.
- [2] React Documentation & Vite.js Guides. *Frontend Tooling & UI Rendering Frameworks*.
- [3] Mozilla Developer Network (MDN). *Canvas API y requestAnimationFrame*. Web documentation.

A. Anexos del Simulador

A.1. ¿Dónde puedo encontrar este código?

El código fuente de este proyecto, incluyendo la lógica de física de colisiones, partición espacial y renderizado a través de HTML5 Canvas, se encuentra disponible para consulta pública en el siguiente repositorio de GitHub: <https://github.com/LuisContla/HormigaDeLangton>.

De igual forma, se puede acceder a la demostración interactiva desplegada en la web y ejecutarla directamente desde el navegador visitando el siguiente enlace: <https://hormiga-de-langton.vercel.app/>.

A.2. Código Fuente Completo

A.2.1. Archivo Principal HTML (index.html)

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <title>hormigadelangton</title>
8   </head>
9   <body>
10    <div id="root"></div>
11    <script type="module" src="/src/main.jsx"></script>
12  </body>
13 </html>
```

A.2.2. Lógica del Componente GUI (LangtonsAnt.jsx)

```
1 import "../src/assets/script.js";
2 import { Crown, HardHat, Heart, Swords, Square } from '../components/Icons.jsx';
3
4 function LangtonsAnt() {
5   return (
6     <div className="contenedor-principal">
7       {/*          HEADER          */}
8       <div className="titulo">
9         <h1> Simulaci n: Hormiga de Langton</h1>
10        <p>M ltiples tipos de hormigas con reglas complejas</p>
11      </div>
12
13      {/*          TOOLBAR          */}
14      <div className="toolbar">
15        <div className="toolbar-group">
16          <button id="generateRandomBtn" className="btn-generate" title="Generar poblaci n aleatoria">
17            Poblar
18          </button>
```

```

19         <button id="toggleGame" className="btn-success" title="Iniciar
/Pausar simulaci n">
20             Iniciar
21         </button>
22         <button id="nextGenerationBtn" className="btn-outline" title="
Avanzar una iteraci n">
23             Paso
24         </button>
25         <button id="resetBtn" className="btn-outline" title="Reiniciar
simulaci n">
26             Reset
27         </button>
28     </div>
29
30     <div className="toolbar-divider" />
31
32     <div className="toolbar-group">
33         <span className="speed-label">Velocidad (ms):</span>
34         <div className="speed-controls">
35             <button id="minSpeed" className="btn-outline btn-small"
title="Velocidad m nima (10ms)">min</button>
36             <button id="decreaseSpeed" className="btn-outline btn-small"
title="Reducir velocidad"> </button>
37             <input type="number" id="speedInput" defaultValue={50} min
={10} step={10} readOnly />
38             <button id="increaseSpeed" className="btn-outline btn-small"
title="Aumentar velocidad">+</button>
39             <button id="maxSpeed" className="btn-outline btn-small"
title="Velocidad m xima (1000ms)">m x </button>
40         </div>
41     </div>
42 </div>
43
44     {/ *          CONTENIDO PRINCIPAL          */}
45     <div className="main-content">
46
47         {/ *          CANVAS          */}
48         <div className="juego-contenedor">
49             <div className="instrucciones-wrapper">
50                 <button className="instrucciones-toggle" title="
Instrucciones del rat n" aria-label="Ver instrucciones">?</button>
51                 <div className="instrucciones-popup">
52                     <strong>Controles del rat n:</strong>
53                     <ul>
54                         <li><strong>Clic Izquierdo:</strong> Dibuja la hormiga o
celda seleccionada. Si ya hay una hormiga, la cambia de tipo.</li>
55                         <li><strong>Clic Derecho:</strong> Elimina la hormiga
debajo del cursor.</li>
56                     </ul>
57                 </div>
58             </div>
59             <canvas id="gameCanvas" />
60         </div>
61
62         {/ *          PANEL LATERAL          */}
63         <aside className="panel-lateral">
64
65             {/ * Estad sticas */}

```

```

66     <div className="seccion">
67         <div className="seccion-header">
68             <span className="seccion-icon">          </span> Estadísticas
69         </div>
70         <div className="stats-grid">
71             <div className="stat-item">
72                 <span className="stat-label">Generaciones</span>
73                 <span className="stat-value" id="generationCounter">0</
span>
74             </div>
75             <div className="stat-item">
76                 <span className="stat-label">Población</span>
77                 <span className="stat-value" id="aliveCounter">0</span>
78             </div>
79
80             <div className="stat-divider" />
81
82             <div className="stat-item stat-reina">
83                 <span className="stat-label">          Reinas</span>
84                 <span className="stat-value" id="countReinas">0</span>
85             </div>
86             <div className="stat-item stat-trabajadora">
87                 <span className="stat-label">          Trabajadoras</span>
88                 <span className="stat-value" id="countTrabajadoras">0</
span>
89             </div>
90             <div className="stat-item stat-reproductora">
91                 <span className="stat-label">          Reproductoras</span>
92                 <span className="stat-value" id="countReproductoras">0</
span>
93             </div>
94             <div className="stat-item stat-soldado">
95                 <span className="stat-label">          Soldados</span>
96                 <span className="stat-value" id="countSoldados">0</span>
97             </div>
98
99             <div className="stat-divider" />
100
101             <div className="stat-item stat-nacimientos">
102                 <span className="stat-label">Nacimientos</span>
103                 <span className="stat-value" id="countNacimientos">0</
span>
104             </div>
105             <div className="stat-item stat-decesos">
106                 <span className="stat-label">Decesos</span>
107                 <span className="stat-value" id="countDecesos">0</span>
108             </div>
109             <div className="stat-item stat-conflictos">
110                 <span className="stat-label">Conflictos</span>
111                 <span className="stat-value" id="countConflictos">0</
span>
112             </div>
113             <div className="stat-item">
114                 <span className="stat-label">Densidad</span>
115                 <span className="stat-value" id="densityStat">0%</span>
116             </div>
117
118             <div className="stat-divider" />

```

```

119         <div className="stat-item full-width">
120             <span className="stat-label">Ratio Naci/Decesos</span>
121             <span className="stat-value" id="ratioStat">1.00</span>
122         </div>
123     </div>
124 </div>
125 </div>
126
127 { /* Pincel Manual */ }
128 <div className="seccion">
129     <div className="seccion-header">
130         <span className="seccion-icon">          </span> Pincel Manual
131     </div>
132     <div className="brush-selector" id="brushContainer">
133         <button className="brush-btn active" data-brush="celda"
134         title="Invertir Celda"><Square size={16} /></button>
135         <button className="brush-btn" data-brush="0" style={{
136         backgroundColor: 'var(--color-reina)', }} title="Reina"><Crown size
137         ={16} /></button>
138         <button className="brush-btn" data-brush="1" style={{
139         backgroundColor: 'var(--color-trabajadora)', }} title="Trabajadora"><
140         HardHat size={16} /></button>
141         <button className="brush-btn" data-brush="2" style={{
142         backgroundColor: 'var(--color-reproductora)', }} title="Reproductora"
143         ><Heart size={16} /></button>
144         <button className="brush-btn" data-brush="3" style={{
145         backgroundColor: 'var(--color-soldado)', color: '#0b1120', }} title="
146         Soldado"><Swords size={16} /></button>
147     </div>
148 </div>
149
150 { /* Toggles */ }
151 <div className="seccion">
152     <div className="toggles-row">
153         <div className="toggle-item">
154             <label htmlFor="toroidalCheck">          Mundo Toroidal</
155             label>
156             <label className="switch">
157                 <input type="checkbox" id="toroidalCheck"
158                 defaultChecked />
159                 <span className="slider round" />
160             </label>
161         </div>
162         <div className="toggle-item">
163             <label htmlFor="showPathCheck">          Celdas Visitadas</
164             label>
165             <label className="switch">
166                 <input type="checkbox" id="showPathCheck"
167                 defaultChecked />
168                 <span className="slider round" />
169             </label>
170         </div>
171     </div>
172 </div>
173
174 { /*          COLAPSABLE: Casos de Estudio (Reporte)
175 */ }
176 <details className="seccion-colapsable" open>

```

```

163         <summary> Casos de Estudio (Reporte)</summary>
164         <div className="seccion-body">
165             <p className="seccion-texto-caso" style={{ fontSize: '0.78
rem', color: 'var(--text-muted)', marginBottom: '4px', lineHeight: '
1.4', textAlign: 'justify' }}>
166                 Estas configuraciones corresponden a los casos de
estudio y pruebas m s representativos descritos en el reporte de
estabilizaci n del ecosistema.
167             </p>
168             <button id="btnPreloadSobrepoblacion" className="btn-
preload" title="Escenario de Sobrepoblaci n (Prueba 1)">
169                 Sobrepoblaci n (Prueba 1)
170             </button>
171             <button id="btnPreloadAislamiento" className="btn-preload"
title="Escenario de Aislamiento (Prueba 5)">
172                 Aislamiento (Prueba 5)
173             </button>
174             <button id="btnPreloadCercania" className="btn-preload"
title="Escenario de Cercan a a la Estabilidad (Prueba 15)">
175                 Cercan a a Estabilidad (Prueba 15)
176             </button>
177             <button id="btnPreloadEstabilidad" className="btn-preload"
title="Escenario de Estabilidad Din mica (Prueba 20)">
178                 Estabilidad Din mica (Prueba 20)
179             </button>
180         </div>
181     </details>
182
183     { /* COLAPSABLE: Configuraci n del Grid */
}
184     <details className="seccion-colapsable">
185         <summary> Configuraci n del Grid</summary>
186         <div className="seccion-body">
187             <div className="config-row">
188                 <label htmlFor="inputRows">Filas:</label>
189                 <input type="number" id="inputRows" min={10}
defaultValue={100} step={10} />
190             </div>
191             <div className="config-row">
192                 <label htmlFor="inputCols">Columnas:</label>
193                 <input type="number" id="inputCols" min={10}
defaultValue={100} step={10} />
194             </div>
195             <div className="config-row">
196                 <label htmlFor="inputCellSize">Tama o de celdas:</label>
197                 <input type="number" id="inputCellSize" min={2}
defaultValue={6} />
198             </div>
199             <button id="updateSizeBtn" className="btn-apply">Aplicar
Tama o </button>
200         </div>
201     </details>
202
203     { /* COLAPSABLE: Poblaci n Inicial */
}
204     <details className="seccion-colapsable">
205         <summary> Poblaci n Inicial</summary>
206         <div className="seccion-body">

```

```

207         <div className="config-row">
208             <label htmlFor="inputDensidadTotal">Densidad Total (%)
209             :</label>
210             <input type="number" id="inputDensidadTotal" min="1" max
211             ="100" defaultValue="50" />
212         </div>
213         <div className="section-subtitle">Distribuci n de tipos
214         (%):</div>
215         <div className="ant-density">
216             <div className="ant-type-indicator">
217                 <input type="color" id="colorReina" defaultValue="#
218                 FF0000" disabled style={{ padding: 0, width: 16, height: 16 }} />
219                 <label><Crown size={14} style={{ marginRight: 3 }} />
220                 Reinas:</label>
221             </div>
222             <input type="number" id="inputProbReina" min="0" max="
223             100" defaultValue="10" />
224             </div>
225             <div className="ant-density">
226                 <div className="ant-type-indicator">
227                     <input type="color" id="colorTrabajadora" defaultValue
228                     ="#0000FF" disabled style={{ padding: 0, width: 16, height: 16 }} />
229                     <label><HardHat size={14} style={{ marginRight: 3 }}
230                     /> Trabajadoras:</label>
231                 </div>
232                 <input type="number" id="inputProbTrabajadora" min="0"
233                 max="100" defaultValue="35" />
234                 </div>
235                 <div className="ant-density">
236                     <div className="ant-type-indicator">
237                         <input type="color" id="colorReproductora"
238                         defaultValue="#FF69B4" disabled style={{ padding: 0, width: 16,
239                         height: 16 }} />
240                         <label><Heart size={14} style={{ marginRight: 3 }} />
241                         Reproductoras:</label>
242                     </div>
243                     <input type="number" id="inputProbReproductora" min="0"
244                     max="100" defaultValue="25" />
245                     </div>
246                     <div className="ant-density">
247                         <div className="ant-type-indicator">
248                             <input type="color" id="colorSoldado" defaultValue="
249                             #00FF00" disabled style={{ padding: 0, width: 16, height: 16 }} />
250                             <label><Swords size={14} style={{ marginRight: 3 }} />
251                             Soldados:</label>
252                         </div>
253                         <input type="number" id="inputProbSoldado" min="0" max="
254                         100" defaultValue="30" />
255                         </div>
256                     </div>
257                 </details>
258
259                 { /*          COLAPSABLE: Nacimientos          */ }
260                 <details className="seccion-colapsable">
261                     <summary>          Nacimientos (Probabilidad %)</summary>
262                     <div className="seccion-body">
263                         <div className="ant-density">

```

```
248         <label><Crown size={14} style={{ marginRight: 3 }} />
Reinas:</label>
249         <input type="number" id="inputNacReina" min="0" max="100"
" defaultValue="25" />
250     </div>
251     <div className="ant-density">
252         <label><HardHat size={14} style={{ marginRight: 3 }} />
Trabajadoras:</label>
253         <input type="number" id="inputNacTrabajadora" min="0"
max="100" defaultValue="15" />
254     </div>
255     <div className="ant-density">
256         <label><Heart size={14} style={{ marginRight: 3 }} />
Reproductoras:</label>
257         <input type="number" id="inputNacReproductora" min="0"
max="100" defaultValue="45" />
258     </div>
259     <div className="ant-density">
260         <label><Swords size={14} style={{ marginRight: 3 }} />
Soldados:</label>
261         <input type="number" id="inputNacSoldado" min="0" max="
100" defaultValue="15" />
262     </div>
263 </div>
264 </details>
265
266 </aside>
267 </div>
268 </div>
269 );
270 }
271
272 export default LangtonsAnt;
```

A.2.3. Estilos Globales y UI (index.css)

El código de estilos CSS es demasiado extenso para ser incluido en su totalidad dentro de este reporte, ya que incrementaría considerablemente la cantidad de páginas. En su lugar, el código fuente completo de `index.css` puede ser consultado directamente en el siguiente enlace del repositorio de GitHub: <https://github.com/LuisContla/HormigaDeLangton/tree/main/src>.